

LARAVEL AFTER DEPLOY

ARCHITECTURE, PERFORMANCE,
AND OPERATIONS AT SCALE

Nuruzzaman Milon



Laravel After Deploy

Architecture, Performance, and Operations at Scale

Nuruzzaman Milon

To my family, for making every effort worthwhile.

Preface

Who this book is for

This book is written for a mid-to-senior Laravel engineer who already knows how to build a Laravel application and is now being asked to keep one alive - the person a growing team hands the architecture decisions, the performance incident, and the migration that has to happen without a maintenance window. It assumes you already know how routing, Eloquent, queues, and testing work in Laravel; it doesn't stop to explain the framework. Every page is instead about the layer of decisions that sits on top of it: how you structure a codebase as more than one team starts touching it, how you keep it fast once traffic outgrows what fits comfortably on one database, how you keep it correct when the same request can arrive twice, how you keep it available when a dependency you don't control goes down, and how you run all of that at 3 a.m. on a Tuesday without it becoming a story you're still telling a year later.

If you're looking for a Laravel tutorial - how to define a route, how to write a migration, how Eloquent relationships work - this isn't that book, and there are excellent ones already. If you're the engineer who's outgrown those books and is now staring at a monorepo, a queue backlog, or a balance that doesn't add up, keep reading.

How to read it

The book is organized into eight parts, and the parts are ordered by dependency rather than importance, so reading front to back works, but it isn't the only sane way through it. Part 1 (Foundations) and Part 2 (Performance Engineering) build vocabulary that later parts assume - "shared kernel," "N+1," "backfill," and so on. Part 5 (Observability & Monitoring) is worth reading earlier than its page number suggests, because "correlation ID," "SLI/SLO/SLA," and "burn rate" get reused heavily in Parts 4, 6, and 7. Beyond that, Parts 3 through 8 are largely independent of each other, so if you picked this book up because of a live incident with your queues, your database failover, or your CI pipeline, it's reasonable to skip straight to the relevant chapter. Chapter 34, the capstone, assumes you've read - or at least skimmed - everything before it: it's a single worked incident that touches nearly every decision in the book, and it's meant to be read last.

Each chapter follows the same shape. It opens with a concrete production failure - drawn from about thirteen years of real incidents, anonymized - then works through the pattern that would have prevented or fixed it, and closes with a short "Chapter recap" you can scan later without rereading the whole chapter. Not every one of those incidents happened in a Laravel project; over those years I've worked across many languages and frameworks, and where the original system wasn't Laravel, I've translated the failure into a Laravel setting so it can be demonstrated in the stack this book is about.

About the examples

Every incident described in this book actually happened in real production systems. The failure modes, the trade-offs, and the fixes are drawn from lived experience over thirteen years as a professional software engineer. What isn't real is any identifying detail: company names, product names, team names, exact numbers, and anything else that could point back to a specific employer or client has been changed, generalized, or recombined. The named fictional businesses that recur throughout the book - Northfield (an order-processing platform), Vaultly (a wallet service), Reelcast (a livestreaming and short-video app), and others - are stand-ins for this kind of company, not case studies of any single one; they exist because those are the systems that reliably produce the failure modes worth discussing, not because they map one-to-one onto any company you could name. Nothing here discloses a trade secret, a proprietary architecture, or confidential information belonging to any organization I've worked with; the anonymization exists specifically to protect that while keeping the underlying lesson intact. The architecture decisions and opinions in these pages are my own.

A note on unfamiliar terms

Abbreviations are spelled out the first time they're used in a chapter, but if you're dropping into a chapter out of order, Appendix A (Glossary) collects the recurring ones - SLO, SLA, SLI, MTTR, RPO/RTO, ADR, and the rest - in one place, each with a pointer back to the chapter that covers it properly.

A connection pooler (PgBouncer for PostgreSQL, ProxySQL for MySQL, or a managed equivalent offered by most cloud database providers) sits between the application and the database, multiplexing many application-side connections onto a smaller pool of real backend connections. It's the standard fix once a horizontally scaled fleet outgrows what the database can hold open directly.

The math is worth doing explicitly, because it rarely feels urgent until it fails outright: twenty application servers, each running twelve PHP-FPM workers plus eight Horizon queue workers for the jobs covered in Chapter 11, is $20 \times (12 + 8) = 400$ potential concurrent connections from a single application - before a second app in the monorepo (Chapter 3) opens its own connections to the same database, and before a rolling deploy briefly runs old and new workers side by side. A `max_connections` setting of 500 has far less headroom than it looks like on paper.

Approach	What it solves	What it doesn't solve
Increase <code>max_connections</code>	Buys headroom short-term	Increases memory pressure per connection; doesn't fix the underlying growth curve
Connection pooler (PgBouncer/ProxySQL)	Decouples app-side connection count from real backend connections	Adds an operational component and a small latency hop
Persistent connections (PDO)	Reduces per-request connect overhead on a single host	Doesn't reduce the <i>number</i> of held connections across a fleet - can make pooling more urgent, not less

Connection pooling is a fleet-level concern, not a per-app config flag - it belongs in the same conversation as horizontal scaling and read replicas (Chapter 21 covers the high-availability side of the data layer in more depth).

Pagination at scale: normal, simple, and cursor

Northfield's admin panel has a searchable orders table. It works fine at ten thousand rows. At two million rows, the page takes eight seconds to load - and profiling shows the slow part isn't the twenty-five rows being displayed, it's the `COUNT(*)` query Laravel's default paginator runs to compute the total page count for the "Page 3 of 84,213" control.

Pagination type	Runs a <code>COUNT(*)</code> ?	Deep-page cost	Supports "jump to page N"?	Best for
<code>paginate()</code>	Yes, every request	Degrades with table size (count + offset scan)	Yes	Small-to-medium tables, UI needs page numbers
<code>simplePaginate()</code>	No	Still degrades with offset depth, but skips the count cost	Next/previous only	Medium tables where total count isn't needed
<code>cursorPaginate()</code>	No	Flat - constant cost regardless of depth, given an index	Next/previous only	Large tables, infinite-scroll or API pagination

Cursor pagination avoids `OFFSET` entirely by encoding the last-seen row's sort key into an opaque cursor and asking for everything that sorts after it - a query an index serves directly, regardless of how deep into the table the user has scrolled:

```
$orders = Order::query()
    →orderByDesc('created_at')
    →orderByDesc('id')
    →cursorPaginate(25);
```

Under the hood, that opaque cursor is just a base64-encoded tuple of the last row's sort columns, and it turns the next page into a comparison the index can seek to immediately, instead of a **LIMIT ... OFFSET** the database has to count past:

```
-- What paginate()/simplePaginate() run for page 84,213 (offset
2,105,300):
SELECT * FROM orders ORDER BY created_at DESC, id DESC LIMIT 25 OFFSET
2105300;

-- What cursorPaginate() runs for the equivalent next page - a seek, not
a scan.
-- Laravel spells the tuple comparison out longhand rather than using
row values:
SELECT * FROM orders
WHERE (created_at < '2026-03-14 09:12:00'
      OR (created_at = '2026-03-14 09:12:00' AND id < 84213))
ORDER BY created_at DESC, id DESC
LIMIT 25;
```

The offset version still has to walk (and discard) 2,105,300 rows before it can return the 25 that matter, even though the database never sends them to the application. The cursor version seeks directly to a known point via the **(created_at, id)** index and reads only the 25 rows it returns - which is why its cost stays flat whether the cursor points at row 25 or row 2,500,000.

The tradeoff is real: cursor pagination can't render "Page 3 of 84,213" or let a user jump straight to page 50, because there's no cheap way to know how many rows precede an arbitrary offset without counting them. For the admin panel, the fix was to drop the total-count display and page-number control in favor of next/previous cursor navigation - a small UX change that turned an eight-second page load back into a two-hundred-millisecond one.

Chapter recap

- Dev-scale seed data hides exactly the problems that matter at production scale - test query performance against realistically sized datasets, not two hundred seeded rows.
- Eager load relations (**with**, **loadMissing**) and project only needed columns (**select**) to eliminate both query-count and I/O blowups in loops;
Model :: preventLazyLoading() turns a missing eager load into a failing test instead of a slow endpoint three months later.
- Index the columns your queries actually filter and sort by, in the order they're used, and don't expect a low-selectivity column to earn its own index.
- **EXPLAIN (type, key, rows, Extra)** turns "this feels slow" into a specific, fixable finding - a full scan, a missing key, or an in-memory sort.
- Connection pooling (PgBouncer/ProxySQL) is what keeps a horizontally scaled fleet from exhausting the database's connection ceiling, independent of query volume.
- Offset-based pagination's **COUNT(*)** and deep-offset scans both degrade as a table grows; cursor pagination stays flat at the cost of losing total counts and arbitrary page jumps - a fair trade for large tables.

Chapter 8 - Safe Schema Evolution at Scale

The migration that only failed in production

On Northfield, a fictional order-processing platform, a routine cleanup task renames a column: `orders.buyer_email` becomes `orders.customer_email`, to match naming used everywhere else in the domain. The migration is small, obviously correct, and reviewed without objection. In staging, against a ten-thousand-row table, it runs in forty milliseconds. In production, against a table with two hundred million rows, the `ALTER TABLE` takes an exclusive lock for twenty-five minutes. Every write to `orders` blocks behind it. The API starts returning timeouts, the on-call engineer gets paged, and the migration that "obviously" worked has to be killed mid-run, leaving the schema in an ambiguous state.

Nothing about the migration was wrong, syntactically. It was tested at the wrong scale, the same failure mode as Chapter 7, applied to schema changes instead of queries. This chapter covers the patterns that make schema changes safe regardless of table size: non-blocking DDL, a backward-compatible rename pattern, batched backfills, and - the part teams forget most often - tracking who else reads your schema before you assume a "safe" change is actually safe for everyone.

Why "fast in staging" doesn't transfer

The cost of a schema change is a function of data volume, not query complexity, and that cost varies by database engine and by the specific operation:

- Adding a nullable column with no default is metadata-only and near-instant on both MySQL (InnoDB, modern versions) and PostgreSQL, regardless of table size.
- Adding a column with a default value, or adding a `NOT NULL` constraint to an existing column, historically required rewriting every row to populate or validate the new value - a cost proportional to table size. Recent versions of both engines have narrowed this (PostgreSQL avoids a full rewrite for constant defaults since version 11; MySQL's behavior depends on the specific `ALGORITHM` used), but the safe assumption is: verify the specific operation against the specific engine version in use, on a copy of production-sized data, before trusting it in staging.

- Renaming a column, dropping a column, or changing a column's type can each range from instant metadata changes to full table rewrites, again depending on engine and version.

The rule that survives all of that engine-specific detail: never trust a migration's timing from a small staging database. Run it against a production-sized copy (a recent snapshot, restored somewhere disposable) before it ships, and know in advance whether the specific operation takes a blocking lock.

Concretely, adding a **NOT NULL** constraint to `orders.customer_email` on PostgreSQL as a single step forces the database to scan and validate every one of two hundred million rows while holding a lock that blocks writes for the duration:

```
// Locks orders for the full validation scan - the whole table, all at
once.
Schema::table('orders', function (Blueprint $table) {
    $table->string('customer_email')->nullable(false)->change();
});
```

Splitting it into "add the constraint without enforcing it yet," "validate it separately," and only then "make the column formally **NOT NULL**" turns one long exclusive lock into a near-instant metadata change plus a scan that only takes a brief lock at the very end:

```

-- Step 1: instant - records the constraint without checking existing
rows.
ALTER TABLE orders ADD CONSTRAINT customer_email_not_null
    CHECK (customer_email IS NOT NULL) NOT VALID;

-- Step 2: scans the table, but only takes a brief lock to flip the
-- constraint's state once the scan finds no violations - not for the
-- duration of the scan itself.
ALTER TABLE orders VALIDATE CONSTRAINT customer_email_not_null;

-- Step 3: after the validated check proves there are no nulls,
PostgreSQL
-- can set the column attribute without repeating the full-table scan.
ALTER TABLE orders
    ALTER COLUMN customer_email SET NOT NULL;

```

Run as separate deploys, the first one is safe to ship immediately; validation and the final **SET NOT NULL** can run once the backfill from the next section has finished, with no code depending on their timing.

Concurrent and non-blocking index creation

Adding an index to a large table is one of the most common migrations to get wrong, because a plain **CREATE INDEX** takes a lock that blocks writes for as long as the index build takes - which, on a two-hundred-million-row table, can be tens of minutes.

PostgreSQL's answer is **CREATE INDEX CONCURRENTLY**, which builds the index without holding a write lock, at the cost of a slower two-pass build and a real failure mode: if it's interrupted, it can leave behind an invalid index that has to be dropped and retried, not resumed.

```

CREATE INDEX CONCURRENTLY idx_orders_customer_created
    ON orders (customer_id, created_at);

```

MySQL's InnoDB engine supports online DDL for many index operations via

`ALGORITHM=INPLACE`, `LOCK=NONE`, which allows concurrent reads and writes during the build - but not every DDL operation supports every algorithm, and the safe habit is to check the specific operation against the running engine version rather than assume.

```
ALTER TABLE orders
  ADD INDEX idx_orders_customer_created (customer_id, created_at),
  ALGORITHM=INPLACE, LOCK=NONE;
```

Laravel's migration files don't manage this distinction automatically - a `Schema::table()` migration using `$table->index()` issues a plain `CREATE INDEX/ADD INDEX` unless the raw SQL or a database-specific driver option is used to request the non-blocking variant. On a large table, that's a decision to make explicitly, not a default to trust:

```
public function up(): void
{
    /*
     * Schema::table()->index() takes a blocking lock for the build -
    fine
     * on a small table, a production incident on a two-hundred-million-
    row one.
    */
    DB::statement(
        'CREATE INDEX CONCURRENTLY idx_orders_customer_created ON orders'
        '(customer_id, created_at)'
    );
}
```

`CREATE INDEX CONCURRENTLY` can't run inside the transaction Laravel wraps each migration in by default, so the migration class also needs `public $withinTransaction = false;` - forgetting that is the most common way this pattern fails silently in a migration file, even though it works fine when run by hand. And because an interrupted concurrent build leaves an unusable index behind rather than rolling back, always check for one before retrying:

```

-- Finds indexes left in an invalid state by an interrupted CONCURRENTLY
build.
SELECT indexrelid::regclass AS index_name
FROM pg_index
WHERE indisvalid = false;

-- Safe to drop and retry - an invalid index is never used by the
planner.
DROP INDEX CONCURRENTLY idx_orders_customer_created;

```

The backward-compatible column rename pattern

Renaming a column directly - `ALTER TABLE orders RENAME COLUMN buyer_email TO customer_email` - is fast as a pure metadata operation on most engines. The real danger isn't the `ALTER` itself; it's that the moment it commits, every piece of code still referencing the old column name breaks atomically, everywhere, at once - including code you don't control (see the next section). The fix is to never do a rename as a single step. Expand it into phases that can each be shipped, verified, and rolled back independently:

Phase	Schema state	Application state	Rollback if something's wrong
1. Add	New column exists, nullable	Not yet used	Drop the new column - no impact
2. Dual-write	Both columns exist	App writes to both columns on every write path; reads still from the old column	Stop dual-writing - old column is still authoritative
3. Backfill	Both columns exist	Batch job populates the new column for existing rows	Backfill is idempotent - safe to re-run or pause

Phase	Schema state	Application state	Rollback if something's wrong
4. Migrate reads	Both columns exist	App reads from the new column; old column no longer read	Revert the read change - old column is still fully populated
5. Stop dual-write	Both columns exist	App writes only to the new column	Revert to dual-writing if any consumer still expects the old column
6. Drop	Old column removed	Fully migrated	Not reversible past this point - this phase only ships once nothing reads the old column

Each phase is a separate, independently deployable change. If phase 4 reveals a bug (the new column has a subtly different meaning than assumed), the rollback is "revert one deploy," not "recover from a completed rename." This is the same expand-and-contract shape used for the shared-package upgrades in Chapter 6 - the pattern generalizes to any change where "old" and "new" need to coexist for a transition window.

Phase 1 is the only phase that touches the database schema directly - it's a plain, additive migration:

```
public function up(): void
{
    Schema::table('orders', function (Blueprint $table) {
        // after() is MySQL-only; omit it on PostgreSQL.
        $table->string('customer_email')->nullable()->after('buyer_email');
    });
}
```

Phase 2's dual-write lives in the model, not in a migration, and is the one line of application code every write path now depends on until phase 5 removes it - with the caveat that "every write path" means every write path that goes through a model instance.

`Order::query()→update( ... )` and anything built on `DB::table('orders')` skip model events entirely, so grep for those before trusting this hook, because phase 4's correctness rests on it:

```
protected static function booted(): void
{
    static::saving(function (Order $order) {
        if ($order→isDirty('buyer_email')) {
            $order→customer_email = $order→buyer_email;
        }
    });
}
```

Phase 6's drop is deliberately its own migration, shipped only after the checklist at the end of this chapter is fully checked off:

```
public function up(): void
{
    Schema::table('orders', function (Blueprint $table) {
        $table→dropColumn('buyer_email');
    });
}
```

Batching large backfills

Phase 3 above - populating the new column for two hundred million existing rows - is itself a hazard if done as one statement:

```
// Don't do this on a large table.
DB::table('orders')→update(['customer_email' =>
DB::raw('buyer_email')]);
```

A single **UPDATE** touching every row holds row locks (and, depending on engine and

isolation level, can hold them for the duration of one very long transaction), competes with live traffic for the same rows, and - on replicated setups - can create significant replication lag, since a replica applies the same enormous write as one unit. The safe alternative is chunking: touch a bounded number of rows per statement, with a small pause between batches to let replicas catch up and to leave room for live traffic.

```
Order :: query()
  →whereNull('customer_email')
  →orderBy('id')
  →chunkById(1000, function ($orders) {
    Order :: query()
      →whereIn('id', $orders→pluck('id'))
      →update(['customer_email' ⇒ DB::raw('buyer_email')]);

    usleep(100_000); // 100ms - bound replication lag and lock
    contention
  });
```

Running this as a resumable Artisan command that processes one bounded batch (tracking the last processed ID) rather than a single migration step means it can be paused, monitored, and restarted without redoing completed work - important for a backfill that might run for hours against a genuinely large table:

```
final class BackfillCustomerEmail extends Command
{
    protected $signature = 'orders:backfill-customer-email {--fresh} {--
dry-run} {--batch=1000}';

    public function handle(): int
    {
        $lastId = $this→option('fresh') ? 0 : (int)
DB::table('backfill_progress')
      →where('name', 'customer_email')
      →value('last_id');

        $dryRun = $this→option('dry-run');
        $orders = Order::query()
      →whereNull('customer_email')
```

```

        →where('id', '>', $lastId)
        →orderBy('id')
        →limit((int) $this→option('batch'))
        →get(['id']);

    if ($orders→isEmpty()) {
        $this→components→info('Backfill complete: no rows remain
with a null customer_email.');
```

return self:: SUCCESS;

```
    }

    $nextLastId = $orders→last()→id;

    if (! $dryRun) {
        // whereNull again, so a concurrent write that already
populated
        // one of these rows isn't overwritten by a stale read.
        Order::query()
            →whereIn('id', $orders→pluck('id'))
            →whereNull('customer_email')
            →update(['customer_email' => DB::raw('buyer_email')]);

        DB::table('backfill_progress')→updateOrInsert(
            ['name' => 'customer_email'],
            ['last_id' => $nextLastId],
        );
    }

    $this→components→info(
        ($dryRun ? '[dry run] ' : '')."Processed {$orders→count()}
rows (last ID: {$nextLastId})."
    );

    return self:: SUCCESS;
}
}

```

Dispatched via `Schedule::command('orders:backfill-customer-email')→everyMinute()→withoutOverlapping()`, this makes progress in small,

- Reconnects need an explicit reconciliation fetch against authoritative origin data, not just a resumed subscription - otherwise clients silently miss whatever was broadcast during the gap.

Chapter 14 - Rate Limiting & Abuse Protection

2 a.m., and the login endpoint won't stop ringing

The graphs look like an attack, because it is one. Login attempts on Kindred, a fictional social platform, have gone from a baseline of a few hundred per minute to over forty thousand, spread across tens of thousands of IP addresses, each one trying a handful of username/password combinations before moving on. It's credential stuffing: a botnet replaying a leaked password list from an unrelated breach, hoping a slice of users reused the same password here.

The naive fix - block anyone who fails login more than five times - makes things worse. The attack is already distributed across so many source IPs that per-IP blocking barely slows it down, while a support queue fills with real users who mistyped a password twice and are now locked out for an hour. The team that survives this incident well isn't the one with the cleverest CAPTCHA; it's the one that had already decided, in calmer times, what a rate limit protects, at what granularity, and what happens to a legitimate request caught in the blast radius.

That's the actual subject of this chapter: rate limiting is not one mechanism, it's a layered set of decisions about *who* you're limiting, *what unit of time* you're limiting them over, and *what you do* when the limit is hit - decisions that are much cheaper to make before an attack than during one.

Two limiter algorithms, and why the difference matters

Laravel ships with one of the two algorithms below, and it's worth knowing which one before you assume an endpoint is protected: the `RateLimiter` facade and the `throttle` middleware both count requests in cache-based fixed windows.

Fixed window counts requests in discrete buckets (e.g. "requests this calendar minute"). It's cheap - one counter, one TTL - but it has a boundary problem: a client can send its full limit at 11:59:59 and its full limit again at 12:00:01, getting two limits' worth of traffic in two seconds.

Token bucket (and its close cousin, the sliding-window log) models a bucket that refills at a steady rate and is drained by each request. It smooths out the boundary problem because there's no reset instant - the bucket's state is continuous. It costs slightly more to compute (you need the last-refill timestamp, not just a count) but it's a much closer match to "how many requests can this client sustainably send," which is usually the thing you actually care about.

	Fixed window	Token bucket / sliding window
Storage cost	One counter + TTL per key	Timestamp + count per key
Boundary burst	Up to 2x limit across a window edge	Bounded by bucket size, not window edge
Good for	Coarse, cheap limits (e.g. "1000 req/hour per API key")	Limits where burst behavior matters (login, payment attempts)
Laravel primitive	<code>RateLimiter::attempt()</code> with default cache-based windows	Custom atomic bucket in Redis or an edge-provider limiter

For most application-level throttles, the fixed window's imprecision doesn't matter - nobody cares if a reporting endpoint allows a small burst across a minute boundary. For anything security-sensitive (login, password reset, payment attempts, OTP verification), the boundary burst is exactly the gap an attacker will find, so it's worth the extra complexity of a token bucket - which you will be building yourself, in Redis or at the edge, because it isn't a mode you can switch the stock limiter into.

Choosing the key: per-user, per-IP, per-action, or a combination

A rate limit is only as good as the key it's keyed on, and this is where most naive implementations fail.

- **Per-IP** limits stop a single machine from hammering an endpoint, but are close to useless against a distributed botnet, and actively harmful behind carrier-grade NAT or corporate proxies, where thousands of real users share one visible IP.
- **Per-user (or per-account)** limits are precise but only apply once you know who the user is - they don't help on an unauthenticated login attempt where the "user" is exactly what's being guessed.
- **Per-action** limits (e.g. "5 password reset emails per hour, regardless of who's asking") protect a shared downstream resource (an email provider, an SMS gateway) from being exhausted by any single actor.
- **Composite keys** - IP *and* the attempted username, or device fingerprint *and* account - are usually where the real protection lives, because they let you set a tight limit on the narrow combination ("this IP trying this specific username") without punishing an entire shared IP range for one bad actor.

The credential-stuffing scenario above is precisely why single-axis keys fail: per-IP limiting is defeated by the botnet's IP diversity, and per-username limiting alone would let the botnet grind through the account list one attempt per username, staying under any reasonable per-account threshold. The fix is layered limits on different keys simultaneously, each cheap on its own, expensive to defeat all at once.

```
// A layered login throttle: IP-wide, then IP+username, then account-
// wide.
public function attempt(Request $request): RedirectResponse
{
    $ip = $request->ip();
    $username = (string) $request->input('email');

    $limits = [
        "login-ip:{$ip}" => 30,           // this IP, any account,
per minute
        "login-ip-user:{$ip}:{$username}" => 5, // this IP against this
account
        "login-user:{$username}" => 10,      // this account, from
anywhere
    ];

    foreach ($limits as $key => $maxAttempts) {
        if (RateLimiter::tooManyAttempts($key, $maxAttempts)) {
            $seconds = RateLimiter::availableIn($key);
        }
    }
}
```

```

        throw ValidationException::withMessages([
            'email' => "Too many attempts. Try again in {$seconds}
seconds.",
        ]->status(429);
    }
}

if (! Auth::attempt($request->only('email', 'password'))) {
    foreach (array_keys($limits) as $key) {
        // The first failed attempt must create the fixed window's
TTL.

        RateLimiter::hit($key, decaySeconds: 60);
    }

    throw ValidationException::withMessages([
        'email' => 'These credentials do not match our records.',
    ]);
}

// Successful logins do not increment any of the failure counters.
return redirect()->intended('/dashboard');
}

```

Hitting the limiter only on *failed* attempts, rather than on every request, is what keeps this from punishing legitimate traffic. A limiter that counts successes too will eventually lock out the user whose only crime is logging in a lot.

Bot and risk scoring: don't let a vendor outage become your outage

Many teams layer a third-party risk-scoring or bot-detection service (device fingerprinting, behavioral scoring, a managed CAPTCHA) on top of application-level rate limits. These are valuable, but they introduce a dependency that can fail independently of your own infrastructure - and a naive integration turns "the risk vendor is slow" into "nobody can log in."

The rule that matters here: **decide, explicitly, what happens when the risk check itself**

fails or times out, rather than letting an exception bubble up and 500 the request. There are two defensible postures, and the right one depends on what the endpoint protects:

Failure mode	Fail closed (block)	Fail open (allow)
Behavior on vendor timeout	Treat as high-risk, block or challenge	Treat as unscored, let the request through
Right for	High-value, low-volume actions (large withdrawals, account recovery)	High-volume, low-marginal-risk actions (viewing a page, browsing a catalog)
Failure mode if wrong	Vendor blip becomes a full outage for real users	A vendor outage becomes a temporary abuse window
Mitigation	Short timeout (100-300ms) + circuit breaker so a slow vendor degrades to "unscored" quickly	Alert loudly on sustained fail-open so someone investigates rather than it becoming permanent

For Kindred's login endpoint, the pragmatic default is: apply the application-level rate limits (which are self-hosted and always available) unconditionally, and treat the risk score as an *additional* signal that can tighten the response (e.g. require a CAPTCHA) but should not be a hard dependency for the base throttle to function. That way a risk-vendor outage degrades the system to "slightly less sophisticated bot detection," not "login is down."

In code, that means a tight timeout on the vendor call and an explicit fail-open branch, rather than letting the exception propagate:

```

public function scoreLogin(string $ip, string $email): ?RiskScore
{
    try {
        $response = $this->http
            request
            →timeout(0.25) // 250ms: fail fast, don't hold up the login
            →post('https://risk-vendor.example.com/v1/score', [
                'ip' => $ip,
                'email' => $email,
            ]);

        if ($response->failed()) {
            // A 4xx/5xx arrives here, not in the catch block below.
            Log::critical('Risk vendor returned an unsuccessful
response', [
                'status' => $response->status(),
            ]);

            return null;
        }

        return RiskScore::fromArray($response->json());
    } catch (ConnectionException $e) {
        /*
        * Fail open: an unscored login still passes the application-
        level
        * throttle above, it only skips the extra CAPTCHA/challenge
        step.
        */
        Log::critical('Risk vendor unavailable, failing open',
['exception' => $e->getMessage()]);

        return null; // caller treats null as "unscored"
    }
}

```

The loud `Log::critical` call is what keeps a fail-open decision visible instead of silently permanent.

Backpressure: what a limit does after it triggers

A rate limit isn't just "reject the request." What you return, and what you do internally when a limit is being approached, is its own design surface:

- **Return 429 Too Many Requests**, not a generic 403 or 500 - well-behaved clients (and API consumers) know to back off on a 429 specifically, and a **Retry-After** header lets them do it without guessing.

Concretely, that's `RateLimiter::availableIn()` feeding straight into the response:

```
public function login(Request $request): JsonResponse
{
    $key = "login-ip:{$request->ip()}";

    if (RateLimiter::tooManyAttempts($key, 30)) {
        $retryAfter = RateLimiter::availableIn($key);

        return response()->json(
            ['message' => "Too many login attempts. Try again in
{$retryAfter} seconds."],
            429,
        )->header('Retry-After', $retryAfter);
    }

    // ...
}
```

- **Shed load progressively, not as a cliff.** If an upstream dependency (a payment gateway, a third-party API) is itself struggling, consider tightening your own limits proactively rather than waiting for it to start timing out request-by-request - this is the same backpressure idea covered for queues in Chapter 11, applied at the edge instead of the worker layer.
- **Distinguish abuse throttling from capacity throttling.** A login rate limit exists to stop abuse and should feel deliberately strict. A general API rate limit protecting your own database from being overwhelmed is a capacity control, and Chapter 15's load-

This is a sample from "Laravel After Deploy: Architecture, Performance, and Operations at Scale".